

NSCLC_Modeling

Setups

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(survminer)
```

Loading required package: ggplot2

Warning: package 'ggplot2' was built under R version 4.4.3

Loading required package: ggpubr

```
library(ggplot2)
library(timeROC)
library(LTRCforests)
```

Attaching package: 'LTRCforests'

The following object is masked from 'package:base':

print

```
library(tidyr)
library(LTRCtrees)
library(survival)
```

Attaching package: 'survival'

The following object is masked from 'package:survminer':

myeloma

```
nsclc <- read.csv("nsclc.csv") # read in the dataset

# Adjusting factor variables
nsclc <- nsclc %>% mutate(
  stage_f = as.factor(stage_f),
  smoke_f = as.factor(smoke_f),
  prior_med_f = as.factor(prior_med_to_msk_f),
  gender = ifelse(GENDER == "Female", 1, 0) # change Female to 1, and male to 0
) %>% select(-c(
  "GENDER", "STAGE_CDM_DERIVED",
  "SMOKING_PREDICTIONS_3_CLASSES", "PRIOR_MED_TO_MSK"
))
```

1. Cox - Frequency Filtering:

1. Drop any gene mutation that occurs in less than $< 3\%$ of the cohort.

```
# Define threshold
threshold <- 0.03

# clinical/demographics columns to be kept
clinical_cols <- c(
```

```

"X", "PATIENT_ID", "age_at_diagnosis", "event",
"entry_time", "exit_time", "smoke_f", "stage_f", "prior_med_to_msk_f",
"prior_med_f", "gender"
)

# select all gene columns
gene_cols <- setdiff(colnames(nsccl), clinical_cols)

# calculate the mutation frequency for all genes
gene_frequencies <- colMeans(nsccl[, gene_cols], na.rm = TRUE)

# identify genes meet or exceed the threshold
genes_to_keep <- names(gene_frequencies[gene_frequencies >= threshold])

# dataset with the clinical columns + the filtered genes
final_cols <- c(clinical_cols, genes_to_keep)
nsccl_filtered <- nsccl[, final_cols]

# check how many genes were dropped vs. kept
cat("Original number of genes:", length(gene_cols), "\n")

```

Original number of genes: 517

```
cat("Genes kept (>= 3%):", length(genes_to_keep), "\n")
```

Genes kept (>= 3%): 63

```
cat("Genes dropped:", length(gene_cols) - length(genes_to_keep), "\n")
```

Genes dropped: 454

cox ph model:

```

library(tidyr)
library(glmnet)

```

Loading required package: Matrix

Attaching package: 'Matrix'

The following objects are masked from 'package:tidyr':

expand, pack, unpack

Loaded glmnet 4.1-10

```
# Drop missing values so x and y matrices align perfectly
nsclc_filtered_complete <- nsclc_filtered %>% drop_na()

# Create the predictor matrix and survival object
x_vars_freq <- nsclc_filtered_complete %>%
  select(-X, -PATIENT_ID, -entry_time, -exit_time, -event)
x_matrix_freq <- model.matrix(~ . - 1, data = x_vars_freq)

y_surv_freq <- Surv(
  time = nsclc_filtered_complete$entry_time,
  time2 = nsclc_filtered_complete$exit_time,
  event = nsclc_filtered_complete$event
)

# Run Cross-Validated LASSO
set.seed(629)
cv_lasso_freq <- cv.glmnet(
  x = x_matrix_freq,
  y = y_surv_freq,
  family = "cox",
  alpha = 1,
  type.measure = "C"
)
```

Extract winning features and isolate just the genes

```
lasso_coefs_freq <- coef(cv_lasso_freq, s = "lambda.min")
active_features_freq <- rownames(lasso_coefs_freq)[which(lasso_coefs_freq != 0)]
selected_genes_freq <- active_features_freq[active_features_freq %in% genes_to_keep]

cat("Frequency Filter + LASSO selected", length(selected_genes_freq), "genes.\n")
```

Frequency Filter + LASSO selected 30 genes.

Post-LASSO Cox Model

```
# Build clean dataset avoiding the dummy-variable trap for clinical factors
final_data_freq <- nsclc_filtered_complete %>%
  select(
    entry_time, exit_time, event,
    age_at_diagnosis, smoke_f, stage_f, prior_med_f, gender,
    all_of(selected_genes_freq)
  )

clean_cox_freq <- coxph(
  Surv(entry_time, exit_time, event) ~ .,
  data = final_data_freq
)

summary(clean_cox_freq)
```

Call:

```
coxph(formula = Surv(entry_time, exit_time, event) ~ ., data = final_data_freq)
```

n= 7470, number of events= 3838

	coef	exp(coef)	se(coef)	z
age_at_diagnosis	-0.009383	0.990661	0.001556	-6.031
smoke_fNever	-0.145262	0.864796	0.045276	-3.208
smoke_fUnknown	0.403448	1.496978	0.068579	5.883
stage_fStage 4	1.085240	2.960151	0.034942	31.058
prior_med_fPrior medications to MSK	0.374406	1.454128	0.044179	8.475
prior_med_fUnknown	0.199240	1.220475	0.054746	3.639
gender	-0.273003	0.761090	0.033570	-8.132
TP53	0.371038	1.449238	0.036107	10.276
ATR	-0.138538	0.870630	0.088989	-1.557
PTPRD	-0.267074	0.765616	0.062648	-4.263
SMARCA4	0.307134	1.359523	0.057057	5.383
PTPRT	-0.153886	0.857370	0.065931	-2.334
NOTCH4	-0.136707	0.872226	0.085820	-1.593
ERBB2	0.153523	1.165934	0.081283	1.889
KRAS	0.071563	1.074185	0.040563	1.764

MTOR	-0.236795	0.789153	0.102882	-2.302
KEAP1	0.372064	1.450726	0.048823	7.621
PAK7	-0.151916	0.859061	0.093935	-1.617
KMT2D	0.133782	1.143143	0.065634	2.038
MED12	-0.176865	0.837893	0.083639	-2.115
ATM	0.133556	1.142885	0.063238	2.112
SETD2	-0.130581	0.877586	0.076190	-1.714
BRCA2	-0.144558	0.865405	0.089508	-1.615
EGFR	-0.284892	0.752095	0.044382	-6.419
RB1	0.116776	1.123867	0.068466	1.706
STK11	0.277459	1.319772	0.050367	5.509
EPHA3	-0.259544	0.771403	0.074029	-3.506
EPHB1	-0.140496	0.868927	0.086715	-1.620
GRIN2A	0.273526	1.314592	0.070314	3.890
KDR	-0.135303	0.873451	0.084958	-1.593
NFE2L2	0.372918	1.451965	0.086552	4.309
TBX3	-0.137826	0.871250	0.088329	-1.560
POLE	-0.164765	0.848093	0.090182	-1.827
ARID1B	-0.161608	0.850775	0.099486	-1.624
PTEN	0.389285	1.475924	0.080961	4.808
ZFH3	-0.159330	0.852715	0.077319	-2.061
EPHA7	-0.170104	0.843577	0.091167	-1.866
Pr(> z)				
age_at_diagnosis	1.63e-09	***		
smoke_fNever	0.001335	**		
smoke_fUnknown	4.03e-09	***		
stage_fStage 4	< 2e-16	***		
prior_med_fPrior medications to MSK	< 2e-16	***		
prior_med_fUnknown	0.000273	***		
gender	4.21e-16	***		
TP53	< 2e-16	***		
ATR	0.119517			
PTPRD	2.02e-05	***		
SMARCA4	7.33e-08	***		
PTPRT	0.019594	*		
NOTCH4	0.111171			
ERBB2	0.058928	.		
KRAS	0.077693	.		
MTOR	0.021357	*		
KEAP1	2.52e-14	***		
PAK7	0.105824			
KMT2D	0.041521	*		
MED12	0.034462	*		

ATM	0.034689 *
SETD2	0.086549 .
BRCA2	0.106304
EGFR	1.37e-10 ***
RB1	0.088081 .
STK11	3.61e-08 ***
EPHA3	0.000455 ***
EPHB1	0.105187
GRIN2A	0.000100 ***
KDR	0.111251
NFE2L2	1.64e-05 ***
TBX3	0.118670
POLE	0.067696 .
ARID1B	0.104283
PTEN	1.52e-06 ***
ZFH3	0.039333 *
EPHA7	0.062061 .

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

	exp(coef)	exp(-coef)	lower .95	upper .95
age_at_diagnosis	0.9907	1.0094	0.9876	0.9937
smoke_fNever	0.8648	1.1563	0.7914	0.9450
smoke_fUnknown	1.4970	0.6680	1.3087	1.7123
stage_fStage 4	2.9602	0.3378	2.7642	3.1700
prior_med_fPrior medications to MSK	1.4541	0.6877	1.3335	1.5857
prior_med_fUnknown	1.2205	0.8194	1.0963	1.3587
gender	0.7611	1.3139	0.7126	0.8129
TP53	1.4492	0.6900	1.3502	1.5555
ATR	0.8706	1.1486	0.7313	1.0365
PTPRD	0.7656	1.3061	0.6772	0.8656
SMARCA4	1.3595	0.7356	1.2157	1.5204
PTPRT	0.8574	1.1664	0.7534	0.9756
NOTCH4	0.8722	1.1465	0.7372	1.0320
ERBB2	1.1659	0.8577	0.9942	1.3673
KRAS	1.0742	0.9309	0.9921	1.1631
MTOR	0.7892	1.2672	0.6450	0.9655
KEAP1	1.4507	0.6893	1.3183	1.5964
PAK7	0.8591	1.1641	0.7146	1.0327
KMT2D	1.1431	0.8748	1.0052	1.3001
MED12	0.8379	1.1935	0.7112	0.9871
ATM	1.1429	0.8750	1.0097	1.2937
SETD2	0.8776	1.1395	0.7559	1.0189

BRCA2	0.8654	1.1555	0.7262	1.0314
EGFR	0.7521	1.3296	0.6894	0.8204
RB1	1.1239	0.8898	0.9827	1.2853
STK11	1.3198	0.7577	1.1957	1.4567
EPHA3	0.7714	1.2963	0.6672	0.8919
EPHB1	0.8689	1.1508	0.7331	1.0299
GRIN2A	1.3146	0.7607	1.1454	1.5088
KDR	0.8735	1.1449	0.7395	1.0317
NFE2L2	1.4520	0.6887	1.2254	1.7204
TBX3	0.8713	1.1478	0.7328	1.0359
POLE	0.8481	1.1791	0.7107	1.0121
ARID1B	0.8508	1.1754	0.7001	1.0339
PTEN	1.4759	0.6775	1.2594	1.7297
ZFHX3	0.8527	1.1727	0.7328	0.9922
EPHA7	0.8436	1.1854	0.7055	1.0086

Concordance= 0.717 (se = 0.004)

Likelihood ratio test= 1894 on 37 df, p=<2e-16

Wald test = 1891 on 37 df, p=<2e-16

Score (logrank) test = 2019 on 37 df, p=<2e-16

cross validation

```
### 4. Cross-Validation (C-Index)
set.seed(629)
K <- 5
n <- nrow(final_data_freq)
fold_id <- sample(rep(1:K, length.out = n))
cv_cindex_freq <- numeric(K)

for(k in 1:K){
  train_data <- final_data_freq[fold_id != k, ]
  test_data <- final_data_freq[fold_id == k, ]

  fit <- coxph(Surv(entry_time, exit_time, event) ~ ., data = train_data)

  test_data$risk_score <- predict(fit, newdata = test_data, type = "lp", na.action = na.pass)

  c_obj <- concordance(
    Surv(entry_time, exit_time, event) ~ risk_score,
    data = test_data,
```



```

    reverse = TRUE
  )

  cv_cindex_freq[k] <- c_obj$concordance
}

cat("Frequency Pipeline Mean C-index:", mean(cv_cindex_freq, na.rm = TRUE), "\n")

```

Frequency Pipeline Mean C-index: 0.7133466

```

cat("Frequency Pipeline SD C-index:", sd(cv_cindex_freq, na.rm = TRUE), "\n")

```

Frequency Pipeline SD C-index: 0.008869384

calibration visualization

```

library(ggplot2)
library(dplyr)
library(survival)

# 1. Define the time horizon (80th percentile of follow-up time)
# Make sure to use the exit_time from your frequency dataset
time_horizon_freq <- quantile(final_data_freq$exit_time, 0.80)

# 2. Calculate predicted survival probabilities at the time horizon
sf_final_freq <- survfit(clean_cox_freq, newdata = final_data_freq)
pred_surv_freq <- summary(sf_final_freq, times = time_horizon_freq)$surv
pred_surv_freq <- as.numeric(pred_surv_freq)

# 3. Add predictions to the dataset and create risk deciles
final_data_freq$pred_surv <- pred_surv_freq
final_data_freq$decile <- ntile(final_data_freq$pred_surv, 10)

# 4. Calculate observed Kaplan-Meier estimates for each decile
calibration_data_freq <- final_data_freq %>%
  group_by(decile) %>%
  group_modify(~ {

    km_fit <- survfit(Surv(entry_time, exit_time, event) ~ 1, data = .x)
  })

```

```

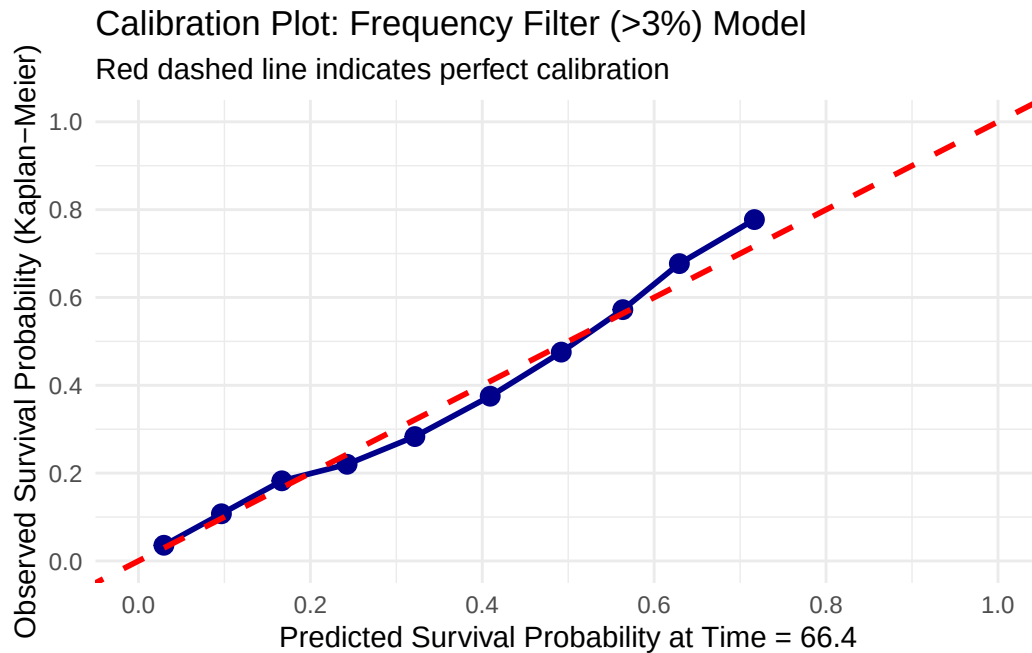
# extend = TRUE ensures we get an estimate even if the curve drops off early
km_summary <- summary(km_fit,
                      times = time_horizon_freq,
                      extend = TRUE)

data.frame(
  mean_pred = mean(.x$pred_surv, na.rm = TRUE),
  km_est = km_summary$surv
)
})

# 5. Plot the calibration curve
ggplot(calibration_data_freq, aes(x = mean_pred, y = km_est)) +
  geom_point(size = 3, color = "darkblue") + # Changed color to easily distinguish from TF-II
  geom_line(color = "darkblue", size = 1) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "red", size = 1) +
  scale_x_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  labs(
    x = paste("Predicted Survival Probability at Time =", round(time_horizon_freq, 1)),
    y = "Observed Survival Probability (Kaplan-Meier)",
    title = "Calibration Plot: Frequency Filter (>3%) Model",
    subtitle = "Red dashed line indicates perfect calibration"
  ) +
  theme_minimal()

```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
 i Please use `linewidth` instead.



```
# Make sure the package is loaded

# 1. Generate the risk scores from your frequency-based model
final_data_freq$risk_score <- predict(clean_cox_freq, type = "lp")

# 2. Define the time horizon (ensuring it matches your calibration plot)
time_horizon_freq <- quantile(final_data_freq$exit_time, 0.80)

# 3. Calculate the Time-dependent ROC
roc_freq <- timeROC(
  T = final_data_freq$exit_time,
  delta = final_data_freq$event,
  marker = final_data_freq$risk_score,
  cause = 1,
  times = time_horizon_freq,
  iid = TRUE # Keeps the data needed to calculate Confidence Intervals
)

# 4. Extract the Area Under the Curve (AUC) and 95% Confidence Intervals
auc_val_freq <- roc_freq$AUC[2]
se_val_freq <- roc_freq$inference$vect_sd_1[2]

ci_lower_freq <- auc_val_freq - 1.96 * se_val_freq
```

```
ci_upper_freq <- auc_val_freq + 1.96 * se_val_freq

cat("Frequency Pipeline - Time Horizon:", round(time_horizon_freq, 2), "\n")
```

Frequency Pipeline - Time Horizon: 66.4

```
cat("Frequency Pipeline - AUC:", round(auc_val_freq, 3), "\n")
```

Frequency Pipeline - AUC: 0.73

```
cat("95% CI: [", round(ci_lower_freq, 3), "-", round(ci_upper_freq, 3), "]\n\n")
```

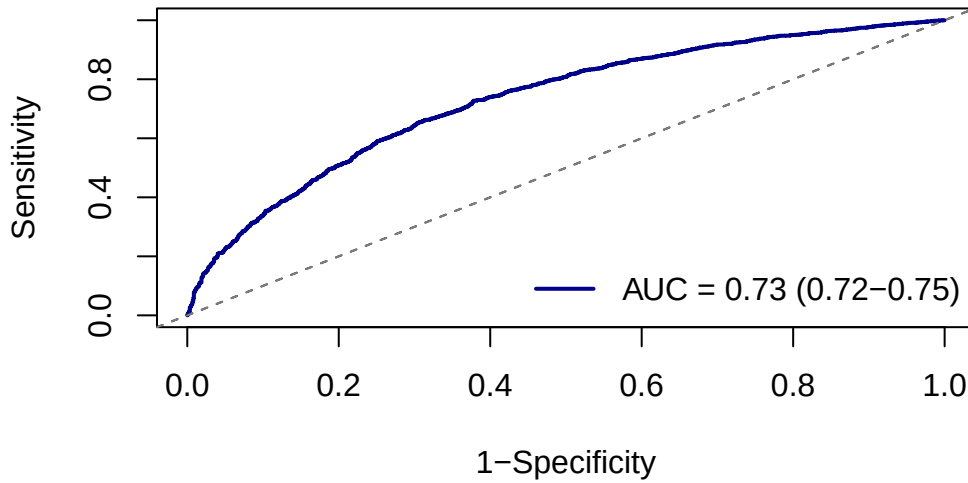
95% CI: [0.715 - 0.746]

```
# 5. Plot the ROC Curve
# Using 'dodgerblue' to match your Frequency calibration plot
plot(roc_freq,
     time = time_horizon_freq,
     col = "darkblue",
     lwd = 2,
     title = FALSE)

# Add reference line and title
abline(0, 1, lty = 2, col = "gray50")
title(main = paste("Frequency Filter (>3%) Model ROC\nat Time =", round(time_horizon_freq, 1)))

# Add legend with the final AUC value
legend("bottomright",
     legend = paste0("AUC = ", round(auc_val_freq, 2),
                     " (", round(ci_lower_freq, 2), "-", round(ci_upper_freq, 2), ")"),
     col = "darkblue",
     lwd = 2,
     bty = "n")
```

Frequency Filter (>3%) Model ROC at Time = 66.4



2. RSF

Train/Test Split (80/20)

```
set.seed(629)
N_total <- nrow(nscclc_filtered_complete)
train_idx <- sample(1:N_total, size = floor(0.8 * N_total))

train_data_rf <- nscclc_filtered_complete[train_idx, ]
test_data_rf <- nscclc_filtered_complete[-train_idx, ]

cat("Train patients:", nrow(train_data_rf), "\n")
```

Train patients: 5976

```
cat("Test patients:", nrow(test_data_rf), "\n")
```

Test patients: 1494

Fit the RSF on TRAIN Data

```
# Define the features to include in the model
clinical_features <- c("age_at_diagnosis", "smoke_f", "stage_f", "prior_med_f", "gender")
all_features <- c(clinical_features, genes_to_keep)

# Construct the formula
rf_formula <- as.formula(
  paste("Surv(entry_time, exit_time, event) ~", paste(all_features, collapse = " + "))
)

# Define mtry
m_try <- max(floor(sqrt(length(all_features)))), 1)

# Fit the Random Forest Model ONLY on the training data
set.seed(629)
rf_model <- ltrcrrf(
  formula = rf_formula,
  data = train_data_rf,
  ntree = 500,
  nodesize = 20,
  mtry = m_try,
  nsplit = 10
)
```

Evaluation on Independent TEST Data

c-index

```
# Define the time horizon based on the training set follow-up
time_horizon_rf <- quantile(train_data_rf$exit_time, 0.80)

# Predict survival probabilities on the TEST set
test_prob_obj <- predictProb(
  rf_model,
  newdata = test_data_rf,
  time.eval = time_horizon_rf
)

# Extract survival and calculate risk on the test set
test_data_rf$pred_surv <- as.numeric(test_prob_obj$survival.probs)
```

```

test_data_rf$risk_score <- 1 - test_data_rf$pred_surv

# c-index
c_obj <- concordance(
  Surv(entry_time, exit_time, event) ~ risk_score,
  data = test_data_rf,
  reverse = TRUE
)

cat("\nOut-of-Sample C-index:", round(c_obj$concordance, 3), "\n")

```

Out-of-Sample C-index: 0.694

Time-Dependent ROC on TEST

```

roc_rf <- timeROC(
  T = test_data_rf$exit_time,
  delta = test_data_rf$event,
  marker = test_data_rf$risk_score,
  cause = 1,
  times = time_horizon_rf,
  iid = TRUE
)

auc_val_rf <- roc_rf$AUC[2]
se_val_rf <- roc_rf$inference$vect_sd_1[2]
ci_lower_rf <- auc_val_rf - 1.96 * se_val_rf
ci_upper_rf <- auc_val_rf + 1.96 * se_val_rf

cat("Out-of-Sample AUC at Time Horizon (", round(time_horizon_rf, 1), "):", round(auc_val_rf, 3), "\n")

```

Out-of-Sample AUC at Time Horizon (66.7): 0.815

```

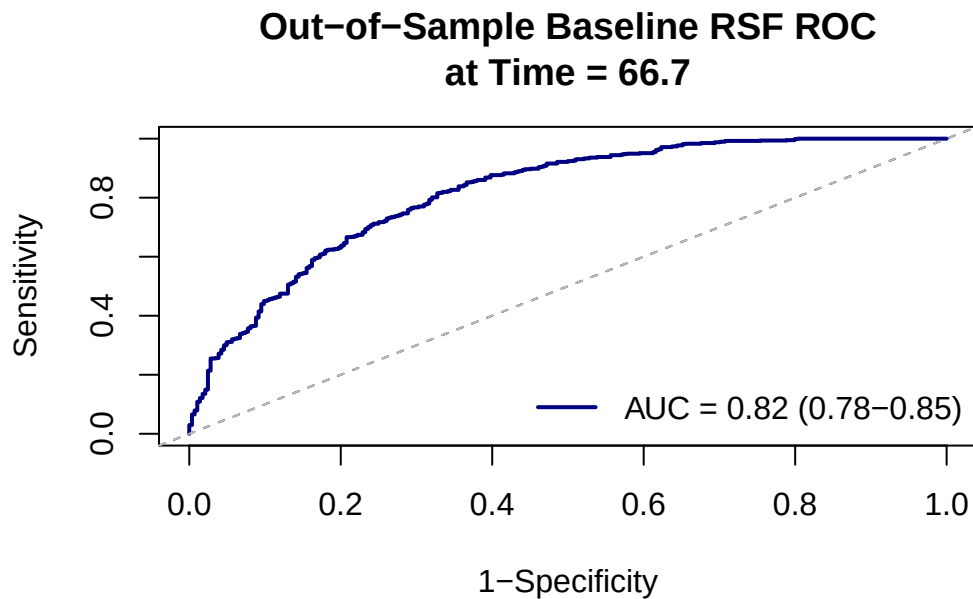
cat("95% CI: [", round(ci_lower_rf, 3), "-", round(ci_upper_rf, 3), "]\n\n")

```

95% CI: [0.784 - 0.846]

```
# Plot the ROC Curve
plot(roc_rf,
     time = time_horizon_rf,
     col = "navy",
     lwd = 2,
     title = FALSE)

abline(0, 1, lty = 2, col = "gray")
title(main = paste("Out-of-Sample Baseline RSF ROC\nat Time =", round(time_horizon_rf, 1)))
legend("bottomright",
      legend = paste0("AUC = ", round(auc_val_rf, 2),
                      " (", round(ci_lower_rf, 2), "-", round(ci_upper_rf, 2), ")"),
      col = "navy",
      lwd = 2,
      bty = "n")
```



Calibration Plot on TEST

```
# Using 5 bins (quintiles) due to the reduced size of the 20% test dataset
test_data_rf$decile <- ntile(test_data_rf$pred_surv, 5)
```



```

calibration_data_rf <- test_data_rf %>%
  group_by(decile) %>%
  group_modify(~ {
    km_fit <- survfit(Surv(entry_time, exit_time, event) ~ 1, data = .x)
    km_summary <- summary(km_fit, times = time_horizon_rf, extend = TRUE)

    data.frame(
      mean_pred = mean(.x$pred_surv, na.rm = TRUE),
      km_est = km_summary$surv
    )
  })

ggplot(calibration_data_rf, aes(x = mean_pred, y = km_est)) +
  geom_point(size = 3, color = "navy") +
  geom_line(color = "navy") +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "red") +
  scale_x_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  labs(
    x = paste("Predicted Survival Probability at Time =", round(time_horizon_rf, 1)),
    y = "Observed Survival Probability (Kaplan-Meier)",
    title = "Out-of-Sample Calibration: Baseline RSF"
  ) +
  theme_minimal()

```

